MINISTRY OF EDUCATION, CULTURE AND RESEARCH
OF REPUBLIC OF MOLDOVA

TECHNICAL UNIVERSITY OF MOLDOVA

FACULTY OF COMPUTERS, INFORMATICS AND
MICROELECTRONICS

DEPARTMENT OF SOFTWARE ENGINEERING AND AUTOMATIC

REPORT ON LABORATORY WORK NO. 4

Course: Operating Systems

The xv6 Shell from the Outside

Done by: st. gr. FAF-243, Andrei Elciscev

Checked by: Patricia Reitman, university assistant
DISA, FCIM, UTM

Chișinău, 2026

Contents

1 Introduction **3**

1.1 Purpose of the Work . . . 3

1.2 Environment . . . 3

2 Part 1 – Fix sleep for the Current xv6 **4**

2.1 Checking the System Call Name . . . 4

2.2 Updating sleep.c . . . 4

2.3 Adding sleep to UPROGS . . . 5

2.4 Building and Testing . . . 5

2.5 Observation . . . 6

3 Part 2 – Bash versus the xv6 Shell **6**

3.1 Experimental Setup . . . 6

3.2 Comparison Table . . . 6

3.3 Bash Results . . . 8

3.4 xv6 Shell Results . . . 8

3.5 Observation 1 – Common and Bash-only Features . . . 9

3.6 Observation 2 – Why ls Stops Working . . . 10

3.7 Observation 3 – Most Useful Missing Feature . . . 12

4 Part 3 – Who Expands *? A Program That Prints Its argv **12**

4.1 Creating args.c for xv6 . . . 12

4.2 Linux Version . . . 12

4.3 Testing args in xv6 . . . 13

4.4 Testing args in Bash . . . 14

4.5 Observation 1 – args *.c "x y" . . . 15

4.6 Observation 2 – Who Expands * and Removes Quotes? . . . 16

4.7 Observation 3 – Extra Spaces . . . 16

5 Part 4 – Read the Shell **16**

5.1 Inspecting user/sh.c . . . 16

5.2 Five Command Types . . . 17

5.3 Where the Commands Are Executed . . . 17

5.4 How cd Is Handled . . . 18

5.5 Observation 3 – Searching for * and $ . . . 18

6 Conclusions **19**

7 Final Answers to Observe Questions **19**

1. Introduction

1.1. Purpose of the Work

The purpose of this laboratory work is to study the xv6 shell from the outside by comparing its behavior with the Bash shell available in Linux. The experiments demonstrate which operations are performed by a shell itself and which are performed by programs or the operating system.

The laboratory also focuses on shell features such as redirection, pipes, command grouping, background execution, globbing, variables, quoting, command history, and command completion. In addition, a small program is developed to print its command-line arguments. This makes it possible to observe directly how Bash and the xv6 shell process wildcards and quotation marks before executing a program.

Finally, the source code of the xv6 shell is inspected to understand its internal command representation and the way different command types are executed.

1.2. Environment

The laboratory work was performed using:

- Linux environment;
- Bash shell;
- xv6-riscv operating system;
- QEMU emulator;
- C programming language;
- GCC compiler;
- Make build system.

The xv6 source tree was located in:

```
~/xv6-riscv
```

Before starting the experiments, the xv6 environment was checked using:

```
os2026-setup.sh --check
```

2. Part 1 – Fix sleep for the Current xv6

2.1. Checking the System Call Name

The first step was to determine which name is currently used by the xv6 source tree for the system call that suspends execution.

The following command was executed:

```
cd ~/xv6-riscv
grep -n "pause\|sleep" user/user.h
```

The result showed that the current xv6 tree uses `pause()` rather than the older `sleep()` name.

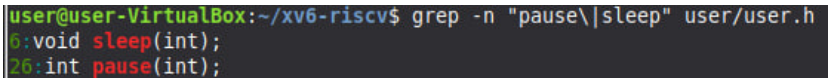


Figure 2.1: The grep result in user.h

2.2. Updating sleep.c

The `sleep` program from the previous laboratory work was updated to use `pause()`.

The resulting program was:

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
    if(argc != 2){
        fprintf(2, "usage: sleep <ticks>\n");
        exit(1);
    }

    pause(atoi(argv[1]));

    exit(0);
}
```

The program checks that exactly one argument is supplied. The argument is converted from a string to an integer and passed to `pause()`.

```
  GNU nano 7.2                                user/sleep.c
#include "kernel/types.h"
#include "user/user.h"
int
main(int argc, char *argv[])
{
if(argc != 2){
fprintf(2, "usage: sleep <ticks>\n");
exit(1);
}
pause(atoi(argv[1]));   // was sleep() before Aug 2025
exit(0);
}
```

Figure 2.2: The updated sleep.c

2.3. Adding sleep to UPROGS

The `sleep` program was also checked in the `UPROGS` section of the xv6 Makefile.

The corresponding entry is:

```
$U/_sleep\
```

The backslash must be the final character on the line, and the line itself must begin with a TAB.

2.4. Building and Testing

The xv6 system was started with:

```
make qemu
```

Inside xv6, the program was tested with:

```
$ sleep 20
$ sleep
usage: sleep <ticks>
```

The first command waits for the specified number of ticks. The second command demonstrates the argument validation performed by the program.

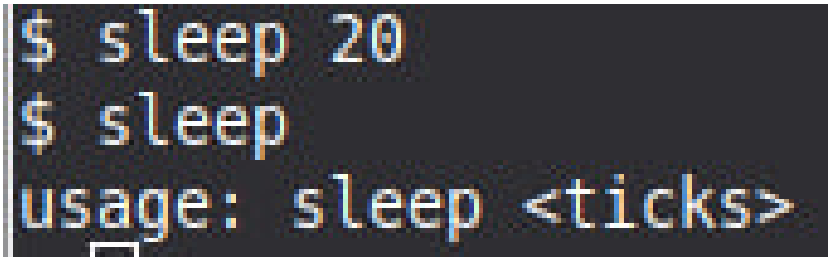


Figure 2.3: sleep running in xv6, with and without an argument

2.5. Observation

The program successfully compiled and executed on the current xv6 version. The important change compared with the older implementation is the use of `pause()` instead of `sleep()`.

This demonstrates that user programs in xv6 depend directly on the system call interface provided by the particular xv6 source tree.

3. Part 2 – Bash versus the xv6 Shell

3.1. Experimental Setup

For the Bash experiments, an empty test directory was created:

```
mkdir ~/shtest
cd ~/shtest
touch README
```

The same commands were then executed in Bash and in the xv6 shell.

The purpose was to compare the features implemented by both shells.

3.2. Comparison Table

The following table presents the expected behavior of the commands before executing them in both shells.

Command / Feature	**Bash**	**xv6 shell**
`echo hello > f; cat < f`	**Prediction:** prints `hello`. Bash supports output and input redirection.	**Prediction:** prints `hello`. xv6 shell supports basic redirection.
`ls | grep READ`	**Prediction:** prints `README`. Bash supports pipes.	**Prediction:** prints `README`. xv6 shell supports pipes.
`(echo a; echo b) | wc`	**Prediction:** prints line/word/byte counts for the two generated lines.	**Prediction:** grouping with parentheses is not supported by xv6 shell and should fail or be interpreted differently.
`echo one ; echo two`	**Prediction:** prints `one` followed by `two`.	**Prediction:** prints `one` followed by `two`.

Command / Feature	**Bash**	**xv6 shell**
`echo bg & ls`	**Prediction:** `echo bg` runs in the background and `ls` runs without waiting for it.	**Prediction:** `&` creates a background command. The output order may differ.
`ls *`	**Prediction:** Bash expands `*` into matching file names before executing `ls`. It should list `README`.	**Prediction:** xv6 does not perform normal wildcard expansion. The shell passes `*` to `ls`, which may therefore fail to find a file literally named `*`.
`echo $HOME`	**Prediction:** prints the user's home directory.	**Prediction:** $HOME is not expanded because xv6 shell does not implement environment-variable expansion.
`echo "a b"`	**Prediction:** prints `a b` as one argument.	**Prediction:** xv6 shell also removes the quotation marks and passes the text as one argument.
`mkdir d; cd d; ls`	**Prediction:** creates directory `d`, enters it and successfully runs `ls`.	**Prediction:** creates `d` and enters it, but `ls` may stop working because xv6 does not search a PATH and the current directory no longer contains the `ls` executable.
`PATH`	**Prediction:** Bash uses the PATH variable to search for executable commands.	**Prediction:** xv6 shell has no equivalent general PATH-based command search.
Up arrow	**Prediction:** recalls previous commands from command history.	**Prediction:** no command-history functionality.
Tab	**Prediction:** performs command or file-name completion.	**Prediction:** no standard Tab completion.

Command / Feature	**Bash**	**xv6 shell**
Ctrl+R	**Prediction:** searches command history interactively.	**Prediction:** no reverse-history search.

3.3. Bash Results

The commands were first executed in Bash. Bash provides many features directly through the shell, including variable expansion, filename globbing, command history, command completion, and PATH-based executable lookup.

```
user@user-VirtualBox:~/shtest$ echo hello > f ; cat < f
hello
user@user-VirtualBox:~/shtest$ ls | grep READ
README
user@user-VirtualBox:~/shtest$ (echo a; echo b) | wc
      2       2       4
user@user-VirtualBox:~/shtest$ echo one ; echo two
one
two
user@user-VirtualBox:~/shtest$ echo bg &
[1] 3352
bg
user@user-VirtualBox:~/shtest$ ls *
f  README
[1]+  Done                    echo bg
user@user-VirtualBox:~/shtest$ echo $HOME
/home/user
user@user-VirtualBox:~/shtest$ echo "a b"
a b
user@user-VirtualBox:~/shtest$ mkdir d ; cd d ; ls ..
d  f  README
```

Figure 3.1: The commands run in Bash

3.4. xv6 Shell Results

The same commands were executed inside the xv6 environment.

The xv6 shell provides a much smaller set of shell features. It supports basic commands, redirection, pipes, command lists, background commands, and quotation handling, but does not provide many of the interactive and expansion features of Bash.

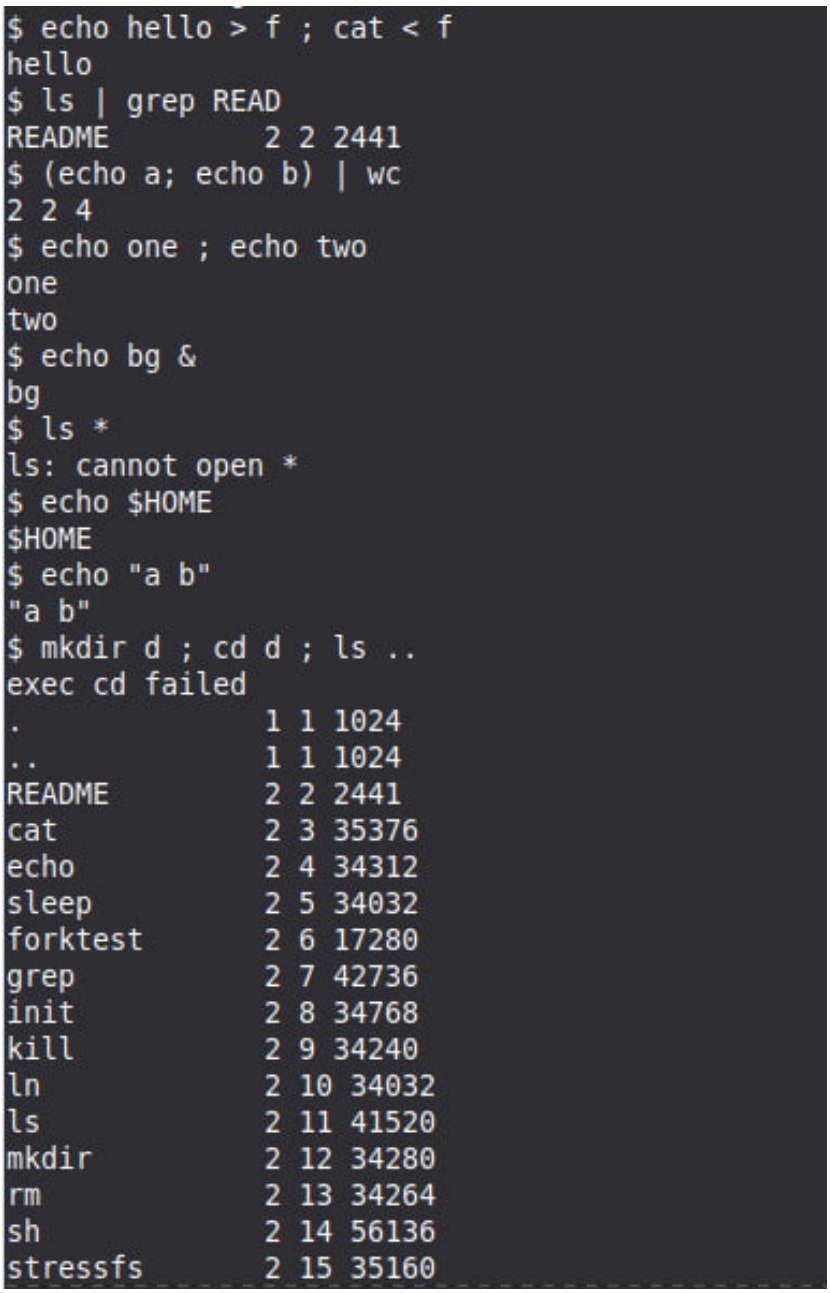


Figure 3.2: The same commands run in xv6

3.5. Observation 1 – Common and Bash-only Features

Both Bash and the xv6 shell support several fundamental shell operations:

- executing programs;
- input and output redirection;
- pipes;
- command sequences;
- background execution;

- changing the current directory;
- basic quotation handling.

Bash provides many additional features, including:

- filename globbing;
- environment-variable expansion;
- PATH-based executable lookup;
- command history;
- reverse-history search;
- Tab completion;
- command grouping with parentheses;
- many additional shell scripting features.

3.6. Observation 2 – Why ls Stops Working

After executing:

```
mkdir d
cd d
ls
```

the `ls` command may stop working inside xv6.

The reason is that the xv6 shell does not perform the same PATH-based search as Bash. The executable files such as `ls` are located in the root directory of the xv6 file system. After entering `d`, simply typing `ls` does not cause the shell to search the root directory for the executable.

The program can still be executed using an explicit path:

```
/ls
```

Therefore, the alternative is to run the executable using its absolute path.

```
$ /ls ..
.               1 1 1024
..              1 1 1024
README          2 2 2441
cat             2 3 35376
echo            2 4 34312
sleep           2 5 34032
forktest        2 6 17280
grep            2 7 42736
init            2 8 34768
kill            2 9 34240
ln              2 10 34032
ls              2 11 41520
mkdir           2 12 34280
rm              2 13 34264
sh              2 14 56136
stressfs        2 15 35160
usertests       2 16 200056
grind           2 17 50504
wc              2 18 36336
zombie          2 19 33648
logstress       2 20 36184
forphan         2 21 35048
dorphan         2 22 34488
sync            2 23 33712
console         3 24 0
f               2 25 6
d               1 26 32
```

Figure 3.3: ls failing after cd d, and the working alternative

3.7. Observation 3 – Most Useful Missing Feature

The feature I would miss most in daily work is command history and completion. These features significantly reduce repetitive typing and make interactive command-line work faster.

The easiest feature to add would probably be a simple command-history mechanism. The shell could store previously entered command strings in memory and allow the user to retrieve them using a key sequence.

4. Part 3 – Who Expands *? A Program That Prints Its argv

4.1. Creating args.c for xv6

A small program was created to display the arguments received by the process.

The xv6 version is:

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
    printf("argc = %d\n", argc);

    for(int i = 0; i < argc; i++)
        printf("argv[%d] = [%s]\n", i, argv[i]);

    exit(0);
}
```

The program was added to the `UPROGS` section of the Makefile:

```
$U/_args\
```

```
user@user-VirtualBox:~/xv6-riscv$ nano user/args.c
user@user-VirtualBox:~/xv6-riscv$ nano Makefile
user@user-VirtualBox:~/xv6-riscv$ nano args_linux.c
user@user-VirtualBox:~/xv6-riscv$ gcc -o args args_linux.c
user@user-VirtualBox:~/xv6-riscv$ ls
args          fs.img  LICENSE   mkfs     test-xv6.py
args_linux.c  kernel  Makefile  README   user
```

Figure 4.1: args.c and the UPROGS line

4.2. Linux Version

The same program was implemented for Linux using the standard C library:

```
#include <stdio.h>

int
main(int argc, char *argv[])
{
    printf("argc = %d\n", argc);

    for(int i = 0; i < argc; i++)
        printf("argv[%d] = [%s]\n", i, argv[i]);

    return 0;
}
```

It was compiled with:

```
gcc -o args args_linux.c
```

4.3. Testing args in xv6

The program was executed using:

```
args one two three
args *.c "x y"
args $HOME
args 'single quoted'
```

The purpose was to determine how the xv6 shell constructs the argument vector.

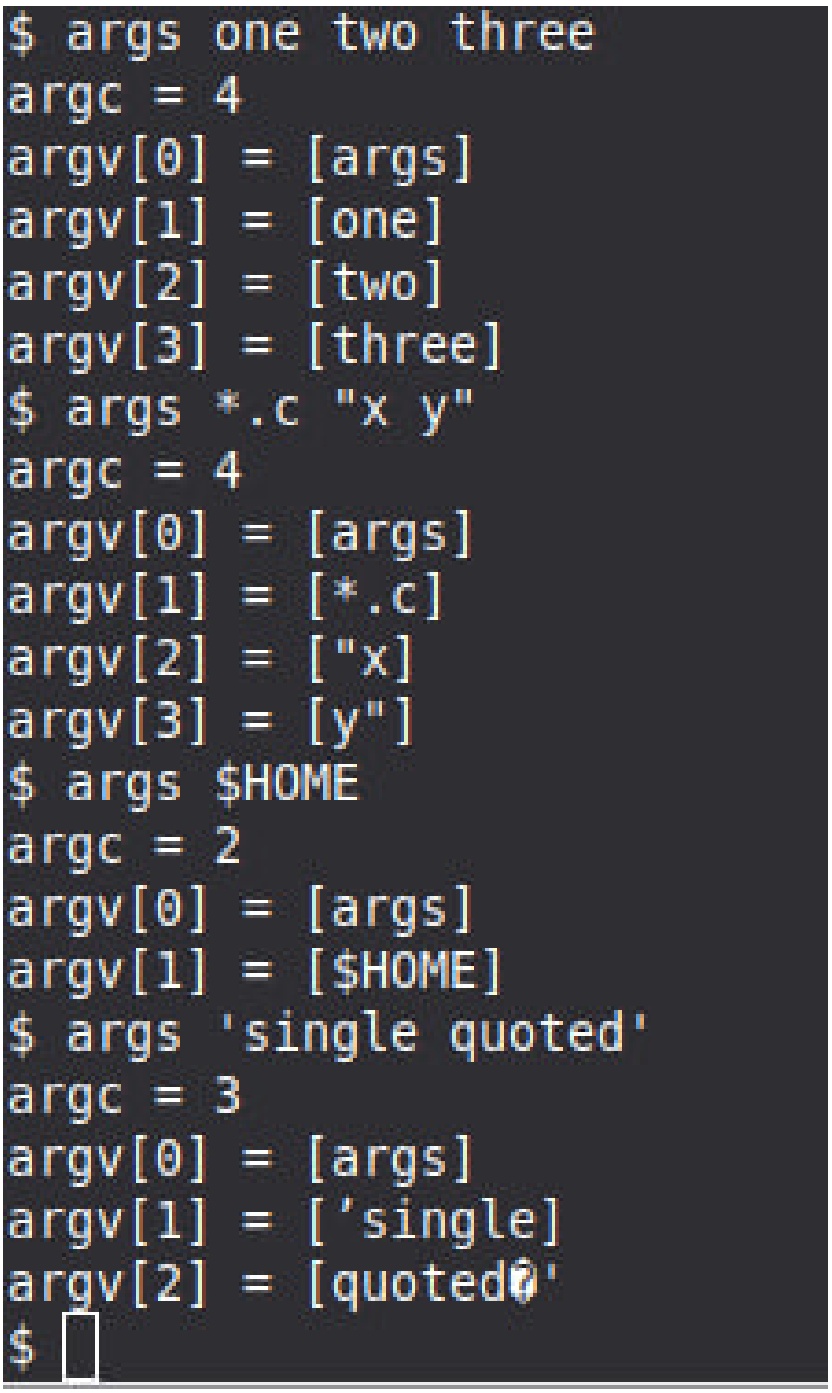


Figure 4.2: args output in xv6

4.4. Testing args in Bash

The same commands were executed in Bash. For the wildcard test, the directory contained several `.c` files.

```
./args one two three
./args *.c "x y"
./args $HOME
./args 'single quoted'
```

```
user@user-VirtualBox:~/xv6-riscv$ ./args one two three
argc = 4
argv[0] = [./args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]
user@user-VirtualBox:~/xv6-riscv$ ./args *.c "x y"
argc = 3
argv[0] = [./args]
argv[1] = [args_linux.c]
argv[2] = [x y]
user@user-VirtualBox:~/xv6-riscv$ ./args $HOME
argc = 2
argv[0] = [./args]
argv[1] = [/home/user]
user@user-VirtualBox:~/xv6-riscv$ ./args 'single quoted'
argc = 2
argv[0] = [./args]
argv[1] = [single quoted]
```

Figure 4.3: args linux output in Bash

4.5. Observation 1 – args *.c "x y"

In Bash, the wildcard `*.c` is expanded by the shell before the program starts.

For example, if the directory contains:

```
args_linux.c
file1.c
file2.c
```

then the command:

```
./args *.c "x y"
```

is transformed approximately into:

```
./args args_linux.c file1.c file2.c "x y"
```

Therefore, the program receives:

```
argc = 5
argv[0] = [./args]
argv[1] = [args_linux.c]
argv[2] = [file1.c]
argv[3] = [file2.c]
argv[4] = [x y]
```

The exact number of arguments depends on the number of `.c` files present in the directory.

In xv6, the shell does not implement wildcard expansion. Therefore, the expected result is:

```
argc = 3
argv[0] = [args]
argv[1] = [*.c]
argv[2] = [x y]
```

Thus, xv6 passes the literal string `*.c` to the program.

4.6. Observation 2 – Who Expands * and Removes Quotes?

The shell performs these operations.

The program itself does not expand wildcards and the kernel does not transform `*.c` into file names. The shell parses the command line before starting the program.

Bash performs filename expansion and removes quotation marks during parsing. The xv6 shell implements a simpler parser and does not perform filename globbing, but it does process quotation marks.

4.7. Observation 3 – Extra Spaces

In:

```
args one two three
```

the spaces separate the command line into three separate arguments.

Multiple spaces between arguments are treated as separators rather than being preserved as part of an argument.

Quotation marks can be used when spaces need to belong to the same argument. For example:

```
args "one two three"
```

produces one argument containing:

```
one two three
```

5. Part 4 – Read the Shell

5.1. Inspecting user/sh.c

The source code of the xv6 shell was examined using the following commands:

```
cd ~/xv6-riscv

wc -l user/sh.c
```

```
grep -n "#define" user/sh.c

grep -n "case " user/sh.c

grep -n "cd" user/sh.c | head
```

The first command shows the approximate size of the shell implementation.

The other commands identify command definitions, command parsing cases, and the special handling of `cd`.

5.2. Five Command Types

The xv6 shell recognizes five fundamental types of commands:

Command type	Character	Meaning
Executable command	None	A normal command that executes a program.
Redirection	<, >	Redirects standard input or output.
Pipe	|	Connects the output of one command to the input of another.
List	;	Executes commands sequentially.
Background	&	Runs a command without waiting for it to finish.

The shell source code contains definitions corresponding to these command structures.

```
user@user-VirtualBox:~/xv6-riscv$ wc -l user/sh.c
499 user/sh.c
user@user-VirtualBox:~/xv6-riscv$ grep -n "#define" user/sh.c
8:#define EXEC  1
9:#define REDIR 2
10:#define PIPE  3
11:#define LIST  4
12:#define BACK  5
14:#define MAXARGS 10
```

Figure 5.1: The command kinds defined in sh.c

5.3. Where the Commands Are Executed

The xv6 shell creates processes and uses system calls such as `fork()`, `exec()`, `wait()`, `pipe()`, `open()`, and `close()` to implement command execution.

Different command structures are handled by different functions in `user/sh.c`.

A normal executable command is ultimately executed using `exec()`, while compound commands create additional processes or file-descriptor relationships.

```
user@user-VirtualBox:~/xv6-riscv$ grep -n "case " user/sh.c
75:  case EXEC:
83:  case REDIR:
93:  case LIST:
101:  case PIPE:
125:  case BACK:
284:  case 0:
286:  case '|':
287:  case '(':
288:  case ')':
289:  case ';':
290:  case '&':
291:  case '<':
294:  case '>':
391:    case '<':
394:    case '>':
397:    case '+': // >>
469:  case EXEC:
475:  case REDIR:
481:  case PIPE:
487:  case LIST:
493:  case BACK:
user@user-VirtualBox:~/xv6-riscv$ grep -n "cd" user/sh.c | head
170:        fprintf(2, "cannot cd %s\n", cmd + 3);
```

Figure 5.2: Where each command kind is run

5.4. How cd Is Handled

The `cd` command is handled specially by the shell.

It cannot be implemented as an ordinary external program because changing the current directory only affects the process that performs the system call.

If the shell created a child process and the child executed `cd`, only the child's working directory would change. The shell itself would remain in the original directory.

Therefore, xv6 handles `cd` directly inside the shell process.

```
user@user-VirtualBox:~/xv6-riscv$ grep -n "cd" user/sh.c | head
170:        fprintf(2, "cannot cd %s\n", cmd + 3);
```

Figure 5.3: The code that handles cd

5.5. Observation 3 – Searching for * and $

The source code of `user/sh.c` was inspected for wildcard and environment-variable expansion.

The xv6 shell does not contain a Bash-like implementation that expands:

```
*.c
```

into a list of matching filenames.

Likewise, it does not implement Bash-style environment-variable expansion such as:

```
$HOME
```

This explains the observations from Part 2. Bash performs these transformations before the program is started, while the xv6 shell passes such characters more directly to the executed program.

6. Conclusions

During this laboratory work, the xv6 shell was studied through direct comparison with Bash and through examination of its source code.

First, the existing `sleep` program was adapted to the current xv6 version by replacing the older `sleep()` system call with `pause()`. The program was successfully compiled and tested.

Second, Bash and xv6 were compared using redirection, pipes, command lists, background execution, globbing, variables, quotation marks, and interactive shell features. The experiment demonstrated that xv6 provides only the fundamental functionality required for a small shell, while Bash provides many additional features such as PATH searching, wildcard expansion, environment-variable expansion, command history, and completion.

Third, the `args` program demonstrated that argument processing takes place before the program receives its `argv` array. In particular, Bash expands wildcards and removes quotation marks, while the xv6 shell does not implement wildcard expansion.

Finally, the source code of `user/sh.c` was examined. The shell represents different command types using separate structures and handles pipes, redirection, lists, and background commands. The special handling of `cd` is necessary because changing the working directory must affect the shell process itself.

Overall, the laboratory demonstrated that a shell is not simply a program launcher. It is a parser and process-management program that interprets command syntax, prepares file descriptors, creates processes, and determines how commands are executed.

7. Final Answers to Observe Questions

Part 1

The current xv6 tree uses `pause()` for the relevant system call. The `sleep` user program was therefore updated to call `pause()` and was successfully tested.

Part 2

1. Both shells provide command execution, redirection, pipes, command sequences, background execution, and basic quotation handling. Bash additionally provides globbing, variable expansion, PATH searching, history, completion, reverse-history search, and more advanced grouping and scripting features.

2. `ls` stops working after entering `d` because xv6 does not perform Bash-like PATH searching. The executable can still be launched using its explicit path, such as `/ls`.

3. The feature I would miss most is command history and completion. A simple command-history mechanism would probably be one of the easiest useful features to add.

Part 3

1. Bash expands `*.c` into the names of matching files, while xv6 passes the literal `*.c`. Therefore, Bash receives one argument for each matching file plus the quoted `x y` argument, while xv6 receives `*.c` and `x y` as separate arguments.

2. The shell performs wildcard expansion and removes quotation marks. The kernel and the executed program do not perform these shell-level transformations.

3. Spaces normally separate arguments. Quotation marks can group text containing spaces into a single argument.

Part 4

1. The five fundamental command types are executable commands, redirection, pipes, lists, and background commands. They are represented by ordinary commands, `</>`, `|`, `;`, and `&`, respectively.

2. `cd` is handled directly by the shell because the working-directory change must affect the shell process itself.

3. xv6's shell does not contain Bash-style implementations of wildcard expansion and environment-variable expansion. This explains why `*.c` and `$HOME` behave differently in xv6 and Bash.